

Piecewise PV Inversion Handbook

The Wu Fortran Reference Pipeline

For the 2025-01-08 California Blocking Case Study

x_yan
pv_inversion/

June 3, 2026

Contents

1	Introduction and Overview	5
1.1	Purpose of this Handbook	5
1.2	Notation Cheat-Sheet	6
1.3	Quick Tour of the Code	6
I	The Wu Fortran Pipeline	7
2	Mathematical Foundations	8
2.1	Coordinates and Constants	8
2.2	Thermodynamic Relations	8
2.3	Non-uniform Vertical Finite Differences	9
2.4	Ertel Potential Vorticity	9
2.5	Linearization about a Mean State	9
2.6	Closure: Coupling $\delta\Phi$ to $\delta\psi$	10
3	The Wu Fortran Reference Solver	11
3.1	Grid and Vertical Levels	11
3.2	The Three Programs and Four Passes	11
3.3	Solver Parameters	12
4	Wu PV to SI Ertel PV: Exact Conversion	13
4.1	Wu PV Definition	13
4.2	True SI Ertel PV	13
4.3	Conversion	14
4.3.1	The Coefficient $c(p)$ is Constant	14
4.4	The Residual $3.5\times$ Factor	14
4.5	Implementation in the Pipeline	15
5	The Wu Pipeline, Step by Step (Benchmark)	16
5.1	Step 05 — Pass A/B: Mean and Event PV, Balanced ψ	16
5.2	Step 06 — Pass C: Total Balanced Inversion (BALNC)	17
5.3	Step 07 — Pass D: Piecewise Perturbation Inversion (BALP)	18
5.4	Step 08 — Parse Outputs and ERA5 Ertel PV	19
5.5	Step 09 — PV Advection by the Induced Winds	20
5.6	Step 10 — Fig. 8 Replica (the Benchmark Result)	21
5.7	SI Output Convention (adopted 2026-06-03)	23

6	Conclusions	24
6.1	Summary	24
6.2	Key Finding: Wu PV to SI Conversion is Exactly 10^{-8}	24
6.3	Status	25
6.4	Non-Dimensional Scales (Reference)	25
A	Notation Glossary	26
B	Public-API Reference	27
B.1	Part I — Wu Fortran reference (wu/)	27
B.1.1	<code>config.py</code> (shared)	27
B.1.2	Fortran programs (wu/fortran/)	27
B.1.3	Step drivers (wu/steps/)	27
B.2	Part II — Wu Python finite-difference port (wu_python/)	27
B.2.1	<code>wu_python/core/grid.py</code>	27
B.2.2	<code>wu_python/core/nondim.py</code>	28
B.2.3	<code>wu_python/core/fd_ops.py</code>	28
B.2.4	<code>wu_python/core/sor_solver.py</code>	28
B.2.5	<code>wu_python/core/pv_calc.py</code>	28
B.2.6	<code>wu_python/core/balance.py</code> (BALNC)	28
B.2.7	<code>wu_python/core/piecewise.py</code> (BALP)	28
B.2.8	<code>wu_python/config.py</code>	28
B.2.9	Step drivers (wu_python/steps/)	29
B.3	Shared utilities (<code>pvtend.sh_ops</code>)	29

List of Figures

5.1	Pass A mean Ertel PV	16
5.2	Event PV anomaly	17
5.3	Balanced mean streamfunction	17
5.4	Pass C refinement	18
5.5	Pass C all levels	18
5.6	Pass D three pieces	19
5.7	Piece 3 small-scale noise	19
5.8	Wu Q vs ERA5 PV	20
5.9	PV advection per piece	21
5.10	Davis 2022 Fig. 8 replica	22

List of Tables

6.1	SI unit conventions for all pipeline outputs.	25
-----	---	----

Chapter 1

Introduction and Overview

1.1 Purpose of this Handbook

This handbook documents **piecewise potential-vorticity inversion** (PPVI) for atmospheric blocking, as implemented in `pv_inversion/`. The goal of the project is to reproduce the diagnostic of Davis et al. (2022, *J. Climate*, their Fig. 8) for the 2025-01-08 California blocking event: partition the upper-level PV anomaly into vertical pieces, invert each piece to a balanced circulation, and quantify how each piece advects PV to sustain the block.

The repository carries **two independent inversion tracks** that share a common ERA5 data-preparation front end:

1. the **Wu Fortran reference** (`pv_inversion/wu/`) — W. Wu’s finite-difference successive-over-relaxation (SOR) solver (`pvpialln_94UV.f`, `qinvert21_94.f`, `qinvertp21_94.f`). This track **works**: it reproduces the Davis 2022 Fig. 8 diagnostic end-to-end and is therefore treated as the *benchmark*. It is the subject of **Part I** (Chapters 3 and 5), where every pass is illustrated with the plot it generates.
2. the **Wu Python finite-difference (FD) port** (`pv_inversion/wu_python/`) — a faithful re-implementation of Wu’s non-dimensional balance system on the same regional 51×87 lat-lon grid, using finite differences and numba-accelerated SOR. This is the subject of **Part II** (??????). It documents the operator mapping, forward PV computation, non-dimensionalization, and the current state of the pure-Python pipeline.

What was retired

Three earlier solvers have been removed from the codebase and from this handbook:

- Two SH solvers — a one-step *Direct Spectral Solve* and an *LGMRES + linearized-Charney* solver (collectively “Family-B”). Their governing closure (the linear f_0 -plane balance) is retained in Section 2.6.
- The **Python spherical-harmonic (SH) BALNC port** — this was a faithful re-implementation on the full-hemisphere SH grid using `pvtend.sh_ops`, but it did not converge and has been superseded by the Wu Python finite-difference port (`wu_python/`).

1.2 Notation Cheat-Sheet

Symbol	Meaning
ψ, Φ	streamfunction ($\text{m}^2 \text{s}^{-1}$), geopotential ($\text{m}^2 \text{s}^{-2}$)
$\bar{\psi}, \bar{\Phi}$	climatological mean (basic) state
$\delta\psi = \psi - \bar{\psi}, \delta\Phi = \Phi - \bar{\Phi}$	anomalies
q	Ertel potential vorticity ($\text{PVU} = 10^{-6} \text{K m}^2 \text{kg}^{-1} \text{s}^{-1}$)
$\bar{q}, q' = q - \bar{q}$	mean and anomalous PV
$\zeta = \nabla^2 \psi$	relative vorticity
$f = 2\Omega \sin \phi, f_0 = f(45^\circ)$	Coriolis parameter, reference value
$\pi = (p/p_0)^\kappa$	Exner function ($p_0 = 10^5 \text{Pa}$, $\kappa = R/c_p$)
$\theta = T(p_0/p)^\kappa$	potential temperature
$A_{\text{avo}} = f + \bar{\zeta}, B_{\text{stb}} = \partial^2 \bar{\Phi} / \partial \pi^2$	avorticity, static stability proxy
∇^2, ∇^{-2}	horizontal Laplacian and its spectral inverse
D_1, D_2	first/second-derivative matrices in π

A more complete glossary appears in [Appendix A](#).

1.3 Quick Tour of the Code

Part I — the Wu Fortran reference (`pv_inversion/wu/`).

Component	Purpose	Reference
<code>wu/fortran/pvpialln_94UV.f</code>	Pass A/B: Ertel PV + balanced ψ, Φ	Chapter 3
<code>wu/fortran/qinvert21_94.f</code>	Pass C: total balanced inversion (BALNC)	Chapter 3
<code>wu/fortran/qinvertp21_94.f</code>	Pass D: piecewise perturbation (BALP)	Chapter 3
<code>wu/steps/05...10/</code>	the step-by-step pipeline + plots	Chapter 5
<code>config.py</code>	grid, levels, solver parameters	Chapter 3

Part II — the Python SH port (`pv_inversion/sh/`).

Module	Purpose	Reference
<code>sh_ppvi/coords.py</code>	vertical Exner grid, D_1, D_2 , Coriolis	Chapter 2
<code>pvtend.sh_ops</code>	SH gradient/Laplacian/Helmholtz (reference only)	??
<code>wu_python/core/pv_calc.py</code>	Ertel PV diagnostic	??
<code>sh_ppvi/nondim.py</code>	Wu non-dimensional scales	??
<code>sh_ppvi/sor.py</code>	BB, BH, BL vertical stencil	??
<code>sh_ppvi/wu_ops.py</code>	Wu-scaled SH operator wrappers	??
<code>sh_ppvi/balnc.py</code>	coupled BALNC ($H-\psi$) Picard solve	??
<code>sh_ppvi/balp.py</code>	piecewise BALP (pending convergence)	??

Part I

The Wu Fortran Pipeline

Chapter 2

Mathematical Foundations

2.1 Coordinates and Constants

The vertical coordinate is the **Exner pressure**

$$\pi \equiv \left(\frac{p}{p_0} \right)^\kappa, \quad p_0 = 10^5 \text{ Pa}, \quad \kappa = \frac{R_d}{c_p} = \frac{287}{1004} \approx 0.2858 \quad (2.1)$$

with $\pi = 1$ at the surface and $\pi \rightarrow 0$ at the model top. On the $K = 10$ ERA5 pressure levels used here we obtain

$$\pi_k = [1.000, 0.978, 0.955, 0.903, 0.864, 0.820, 0.770, 0.709, 0.673, 0.631]. \quad (2.2)$$

Constant	Symbol	Value
Gravity	g	9.80665 m s^{-2}
Dry-air gas constant	R_d	$287 \text{ J kg}^{-1} \text{ K}^{-1}$
Specific heat at p	c_p	$1004 \text{ J kg}^{-1} \text{ K}^{-1}$
$\kappa = R_d/c_p$	κ	0.28586
Earth's rotation	Ω	$7.292 \times 10^{-5} \text{ rad s}^{-1}$
Earth radius	a	$6.371 \times 10^6 \text{ m}$
Reference pressure	p_0	10^5 Pa
Reference Coriolis	f_0	$f(45^\circ) \approx 1.031 \times 10^{-4} \text{ rad s}^{-1}$

2.2 Thermodynamic Relations

Potential temperature (`coords.theta_from_T`):

$$\theta = T \left(\frac{p_0}{p} \right)^\kappa. \quad (2.3)$$

Hydrostatic balance in Exner coordinates (`coords.dpi_phi_from_theta`):

$$\frac{\partial \Phi}{\partial \pi} = -c_p \theta \quad (2.4)$$

which is the dry-static-energy form of hydrostatic balance. It provides the *vertical* link between Φ and θ that will close the inversion.

Coriolis (`coords.coriolis`):

$$f(\phi) = 2\Omega \sin \phi, \quad \frac{df}{dy} = \frac{2\Omega \cos \phi}{a} \equiv \beta. \quad (2.5)$$

2.3 Non-uniform Vertical Finite Differences

ERA5 levels are unevenly spaced in π ; we therefore use three-point Lagrange stencils. At an interior node π_0 with neighbours $\pi_m < \pi_0 < \pi_p$ and spacings $h_m = \pi_0 - \pi_m$, $h_p = \pi_p - \pi_0$:

$$\left. \frac{\partial f}{\partial \pi} \right|_{\pi_0} \approx -\frac{h_p}{h_m(h_m + h_p)} f_m + \frac{h_p - h_m}{h_m h_p} f_0 + \frac{h_m}{h_p(h_m + h_p)} f_p, \quad (2.6)$$

$$\left. \frac{\partial^2 f}{\partial \pi^2} \right|_{\pi_0} \approx \frac{2}{h_m(h_m + h_p)} f_m - \frac{2}{h_m h_p} f_0 + \frac{2}{h_p(h_m + h_p)} f_p. \quad (2.7)$$

At the two boundaries we substitute the second-order one-sided analogues. Assembling the stencils into a matrix gives

$$\frac{\partial f}{\partial \pi} = \mathbf{D}_1 f, \quad \frac{\partial^2 f}{\partial \pi^2} = \mathbf{D}_2 f, \quad \mathbf{D}_1, \mathbf{D}_2 \in \mathbb{R}^{K \times K}. \quad (2.8)$$

These matrices are precomputed once in `coords.D1_MAT`, `coords.D2_MAT`.

2.4 Ertel Potential Vorticity

The hydrostatic Ertel PV reads

$$Q = -g(\zeta + f) \frac{\partial \theta}{\partial p}. \quad (2.9)$$

Substituting eq. (2.4) and the chain rule $\partial/\partial p = (\kappa\pi/p) \partial/\partial \pi$ gives the (ψ, Φ) -only form used throughout the code:

Ertel PV in psi-Phi form

$$Q(\psi, \Phi) = g(\nabla^2 \psi + f) \cdot \underbrace{\frac{\kappa\pi}{p c_p}}_{\equiv \text{coeff}(p)} \cdot \frac{\partial^2 \Phi}{\partial \pi^2} \times 10^6 \text{ [PVU]} \quad (2.10)$$

The factor 10^6 converts SI units to PVU. Equation eq. (2.10) is implemented in both the Wu Python track (`wu_python/core/pv_calc.py`, ??) and the Wu Fortran subroutine `PV_CALC` (Chapter 3).

2.5 Linearization about a Mean State

Decompose $\psi = \bar{\psi} + \delta\psi$, $\Phi = \bar{\Phi} + \delta\Phi$ and Taylor-expand eq. (2.10) to first order, retaining only $\mathcal{O}(\delta)$ terms. After cancellation of the mean contribution $\bar{q} = Q(\bar{\psi}, \bar{\Phi})$:

Linearized Ertel PV

$$q' = g \cdot \text{coeff}(p) \left[\underbrace{(f + \nabla^2 \bar{\psi})}_{A_{\text{avo}}} \frac{\partial^2 \delta\Phi}{\partial \pi^2} + \underbrace{\frac{\partial^2 \bar{\Phi}}{\partial \pi^2}}_{B_{\text{stb}}} \nabla^2 \delta\psi \right] \times 10^6 \quad (2.11)$$

The two basic-state coefficients

$$A_{\text{avo}}(\mathbf{x}) = f + \nabla^2 \bar{\psi} = f + \bar{\zeta}, \quad B_{\text{stb}}(\mathbf{x}) = \frac{\partial^2 \bar{\Phi}}{\partial \pi^2} \quad (2.12)$$

have natural meteorological interpretations: A_{avo} is the *absolute vorticity* of the mean state, and B_{stb} is proportional to the mean *static stability* (via eq. (2.4), $B_{\text{stb}} = -c_p \partial \bar{\theta} / \partial \pi$).

2.6 Closure: Coupling $\delta\Phi$ to $\delta\psi$

Equation eq. (2.11) is one equation in two unknowns $(\delta\psi, \delta\Phi)$. Closure requires a **balance constraint**. Two are used in the code:

(a) **Nonlinear Charney balance** (the mean-state balance of the nonlinear Picard inversion):

$$\nabla^2 \Phi = \nabla \cdot (f \nabla \psi) + 2 J(\psi_x, \psi_y). \quad (2.13)$$

(b) **Linear f_0 -plane balance** (the linear closure referenced by the spherical-harmonic BALNC port, ??):

$$\delta\Phi \approx f_0 \delta\psi \implies \frac{\partial^2 \delta\Phi}{\partial \pi^2} \approx f_0 D_2 \delta\psi \quad (2.14)$$

Substituting eq. (2.14) into eq. (2.11) yields a linear operator that acts *only on* $\delta\psi$:

$$q' = g \text{coeff}(p) [A_{\text{avo}} f_0 D_2 \delta\psi + B_{\text{stb}} \nabla^2 \delta\psi] \times 10^6 \quad (2.15)$$

Equation eq. (2.15) was the central object of the legacy spherical-harmonic piecewise solvers (a *direct* variant dropped the first term; an *LGMRES* variant kept both). Those Family-B solvers have since been retired; the relation survives here as the linear closure cited by the SH BALNC port (??).

Caveat of the f_0 -plane closure

$\delta\Phi \approx f_0 \delta\psi$ is the geostrophic-balance approximation at a reference latitude. It under-estimates the geopotential anomaly at upper levels by a factor that depends on the meridional structure of $\delta\psi$. This is the principal source of the $\approx 0.1 \times$ *amplitude bias* of the SH port vs. the Wu Fortran reference (Chapter 5).

Chapter 3

The Wu Fortran Reference Solver

This chapter describes the working benchmark: W. Wu’s finite-difference piecewise PV-inversion code, ported only in the sense of being *driven* from Python (`wu/steps/`) while the numerical core remains the original Fortran-77. The three programs live in `wu/fortran/` and are compiled with `gfortran -std=legacy -O2 -fno-automatic`.

3.1 Grid and Vertical Levels

The solver operates on a bounded Northern-Hemisphere window (`config.py`):

Quantity	Symbol / code	Value
Longitudes	N_X	87
Latitudes	N_Y	51
Levels	N_W	10
Latitude row	$\phi_i = 85.5 - 1.5 i$	$85.5^\circ \rightarrow -13.5^\circ$ (N→S)
Longitude col	$\lambda_j = -169.5 + 1.5 j$	$-169.5^\circ \rightarrow -40.5^\circ$
Pressure levels	PLEVS [hPa]	1000, 925, 850, 700, 600, 500, 400, 300, 250, 200
Earth radius	$R_\oplus = 2 \times 10^7 / \pi$	6.366×10^6 m
Grid spacing	$\Delta p = \Delta l = R_\oplus \text{rad}(1.5^\circ)$	1.667×10^5 m

Fortran PARAMETER blocks are hard-wired

N_X, N_Y appear as compile-time **PARAMETER**s inside every `.f` file. Changing the grid requires editing and recompiling the Fortran, not just `config.py`. The Python drivers pass solver *parameters* (relaxation factors, iteration caps) over `stdin`; they cannot change the array dimensions.

3.2 The Three Programs and Four Passes

Pass	Program	Subroutine	Produces
A	pvpialln	—	meanq, meanh (mean PV, ψ , Φ)
B	pvpialln	—	event_q.out, event_h.out (event PV, ψ , Φ)
C	qinvert21	BALNC	event_bal.out (total balanced ψ , Φ)
D	qinvert21	BALP	event_pert.out (3-piece perturbation ψ')

Pass A/B — pvpialln. Computes Ertel PV on the interior σ -levels ($K = 2 \dots N_W - 1$, i.e. 925–250 hPa), the balanced streamfunction ψ by relative-vorticity inversion, the geopotential Φ on pseudo-height, and the boundary potential temperature θ at top and bottom. Pass A is run on the 11-day climatological mean state; Pass B on the event state.

Pass C — qinvert21 (BALNC). Solves the full *total* balanced inversion: a 2-D Helmholtz equation for ψ on each σ -level coupled to a 3-D PV– ψ relation under gradient-wind balance, by successive over-relaxation with under-relaxation between ψ and Φ . The non-dimensional form of this coupled system — which the Python SH port re-implements verbatim — is given in ??.

Pass D — qinvertp21 (BALP). Partitions the PV *perturbation* $q' = q_{\text{event}} - q_{\text{mean}}$ into three vertical pieces and inverts each independently to its perturbation streamfunction ψ' :

Piece	Levels K	Pressures
1 (lower)	{1, 2}	1000, 925 hPa
2 (middle)	{3, 4}	850, 700 hPa
3 (upper)	{5, 6, 7, 8, 9}	600, 500, 400, 300, 250 hPa

3.3 Solver Parameters

The relaxation parameters below are passed over **stdin** by the step drivers and are the values that make Pass C/D stable on this grid:

Parameter	Value	Role
OMEGS	1.4	SOR factor for the ψ (streamfunction) sweep
OMEGH	1.4	SOR factor for the Φ/H (geopotential) sweep
PART	0.5	under-relaxation coupling $\psi \leftrightarrow \Phi$
INLIN	0	linear balance (mandatory; see warning)
MAXT	200	outer-iteration cap
THRSH	0.01	convergence threshold

INLIN=0 is mandatory

With the nonlinear balance enabled (INLIN=1) the iteration explodes at the 250 hPa jet. All production passes use the *linear* balance INLIN=0. The same instability re-appears as the dominant obstacle in the SH port (??).

Wu's PV units are solver-internal

Wu's internal Q is *not* in PVU; it is roughly $600\times$ a PVU (an artifact of the non-dimensionalisation, ??). Every physical-unit diagnostic (PV anomaly contours, PV advection) is therefore computed from a *separately* evaluated Ertel PV on the ERA5 fields (Section 5.4), never from Wu's Q directly.

Chapter 4

Wu PV to SI Ertel PV: Exact Conversion

This chapter documents the *exact analytic* conversion from the Wu Fortran PV scalar Q_{Wu} to true SI Ertel potential vorticity. The conversion coefficient is **exactly** 10^{-8} at every pressure level when $\kappa = 2/7$, which is the standard dry-air value used throughout the Wu code. This result was derived on 2026-06-03 and replaces an earlier ad-hoc median-ratio scaling.

4.1 Wu PV Definition

In `pvpialln_94UV.f` (lines 460–468), Ertel-like PV is computed as

$$Q_{\text{Wu}} = -\text{COEF} \cdot \Pi^{-2.5} \left[(f + \zeta) \frac{\partial \theta}{\partial \Pi} + \left(\frac{\partial u}{\partial \Pi} \frac{\partial \theta}{\partial y} - \frac{\partial v}{\partial \Pi} \frac{\partial \theta}{\partial x} \right) \right], \quad (4.1)$$

where

$$\text{COEF} = \frac{10^2 \cdot 10^6 \cdot g \cdot \kappa \cdot c_p^{3.5}}{p_0}, \quad (4.2)$$

$$\Pi(p) = c_p \left(\frac{p}{p_0} \right)^\kappa, \quad (4.3)$$

with numerical constants $g = 9.81 \text{ m s}^{-2}$, $\kappa = 2/7$, $c_p = 1004.5 \text{ J kg}^{-1} \text{ K}^{-1}$, $p_0 = 10^5 \text{ Pa}$ (but see warning below). The “1.E2” in the COEF expression is a legacy over-scaling of no physical significance; it is the conjugate of the 10^{-2} divide applied to PV in the non-dimensional BALNC/BALP solvers, and it cancels exactly in the back-conversion derived below.

4.2 True SI Ertel PV

The standard definition of Ertel PV in isobaric coordinates is

$$\text{PV}_{\text{SI}} = -g \left[(f + \zeta) \frac{\partial \theta}{\partial p} + \frac{\partial u}{\partial p} \frac{\partial \theta}{\partial y} - \frac{\partial v}{\partial p} \frac{\partial \theta}{\partial x} \right], \quad (4.4)$$

with units $\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}$ ($\equiv 10^{-6} \text{ PVU}$).

4.3 Conversion

The two definitions, (4.1) and (4.4), differ only in the vertical coordinate of differentiation: $\partial/\partial\Pi$ in Wu versus $\partial/\partial p$ in SI. The chain rule relates them:

$$\frac{\partial}{\partial p} = \frac{\partial\Pi}{\partial p} \frac{\partial}{\partial\Pi} = \frac{\kappa\Pi}{p} \frac{\partial}{\partial\Pi}. \quad (4.5)$$

The factor $\kappa\Pi/p$ multiplies *both* the vorticity-stability term and the shear term identically, so it factors out of the entire bracket:

$$\begin{aligned} \text{PV}_{\text{SI}} &= Q_{\text{Wu}} \cdot \frac{g}{\text{COEF}} \cdot \frac{\kappa\Pi}{p} \cdot \Pi^{2.5} \\ &= Q_{\text{Wu}} \cdot \frac{g\kappa}{\text{COEF}} \cdot \frac{\Pi^{3.5}}{p} \equiv Q_{\text{Wu}} \cdot c(p). \end{aligned} \quad (4.6)$$

4.3.1 The Coefficient $c(p)$ is Constant

Substituting $\Pi = c_p(p/p_0)^\kappa$ and $\text{COEF} = 10^8 g \kappa c_p^{3.5}/p_0$:

$$\begin{aligned} c(p) &= \frac{g\kappa}{10^8 g \kappa c_p^{3.5}/p_0} \cdot \frac{c_p^{3.5}(p/p_0)^{3.5\kappa}}{p} \\ &= \frac{1}{10^8} (p/p_0)^{3.5\kappa-1}. \end{aligned} \quad (4.7)$$

With $\kappa = 2/7$,

$$3.5\kappa = \frac{7}{2} \cdot \frac{2}{7} = 1 \quad \implies \quad c(p) = \frac{1}{10^8} (p/p_0)^0 = \mathbf{10^{-8}} \quad (4.8)$$

at *every* pressure level. Wu’s choice of the $\Pi^{-2.5}$ weighting was designed precisely so that $\Pi^{3.5}/p$ is pressure-independent: the p dependence in $\Pi^{3.5}$ exactly cancels with the $1/p$ from the chain rule.

Wu Q $\rightarrow$ SI conversion is a universal constant

$$\text{PV}_{\text{SI}} = Q_{\text{Wu}} \times 10^{-8}$$

or

$$Q_{\text{Wu}} = \text{PV}_{\text{SI}} \times 10^8$$

This holds for *any* pressure level when $\kappa = 2/7$. If a different κ were used, the per-level array $c(p) = (g\kappa/\text{COEF}) \cdot \Pi^{3.5}/p$ must be evaluated; the ‘WU2SI’ array in `parse_and_pv.py` implements this general form.

4.4 The Residual $3.5\times$ Factor

Dividing Wu Q by 10^8 converts it to *genuine SI units* ($\text{K m}^2 \text{kg}^{-1} \text{s}^{-1}$). The resulting field, when compared to ERA5’s native Ertel PV, shows a spatially-varying residual factor of $\sim 2\text{--}4\times$ (median $\sim 3.5\times$). This residual is *physical*, not a units problem:

- Wu uses a 5-point finite-difference Laplacian on a 1.5° grid; ERA5 computes PV from its native spectral representation.
- Wu computes $\partial\theta/\partial\Pi$ from 3-point centred differences on 8 unevenly-spaced σ -levels; ERA5 has 137 model levels.

- The static stability $\partial\theta/\partial p$ is sensitive to vertical resolution; coarse differencing systematically underestimates it by $\sim 2\text{--}3\times$.

The median ratio of $Q_{\text{Wu}} \times 10^{-8}$ to ERA5 PV can be reported as a diagnostic of the discretisation bias, but it must *not* be used to scale the Wu PV field. The correct conversion is the analytic 10^{-8} .

4.5 Implementation in the Pipeline

In `wu/steps/08_parse_outputs/parse_and_pv.py`:

```

1 # Exact Wu Q -> SI Ertel PV per-level coefficient
2 PLEVS_PA = np.asarray(WU_LEVS) * 100.0
3 PI_LEV   = CP * (PLEVS_PA / P0) ** KAP
4 COEF_WU  = 1.0e2 * 1.0e6 * 9.81 * KAP * CP**3.5 / P0
5 WU2SI    = (G * KAP / COEF_WU) * PI_LEV**3.5 / PLEVS_PA # ~1e-8
6 Q_wu_si  = Q_wu * WU2SI[k250] # SI [K.m2.kg-1.s-1]
7 Q_wu_pvu = Q_wu_si * 1e6 # PVU for display

```

The $c(p) = 10^{-8}$ constant is also documented in `config.py` section 7 alongside the full Wu non-dimensional scale set (DPI, LL, FF, THO, HND, QCONST, PI_WU, PIF, BB/BH/BL).

COEF uses g

The Wu Fortran hardcodes $g = 9.81$ (line 460), while the standard SI value is $g = 9.80665 \text{ m s}^{-2}$. The Python coefficients match the Fortran's 9.81 for exact numeric consistency. The difference is $< 0.04\%$ and is absorbed by the residual discretisation bias anyway.

Chapter 5

The Wu Pipeline, Step by Step (Benchmark)

This chapter walks through the six executable steps in `wu/steps/` that take the raw Wu Fortran output to the Davis 2022 Fig. 8 diagnostic. Each step is illustrated with the figure it generates (regenerated for this handbook into `handbook/figs/wu/`). This pipeline is the project **benchmark**: the SH port of Part II is judged against it.

5.1 Step 05 — Pass A/B: Mean and Event PV, Balanced ψ

Step 05 compiles the three Fortran programs and runs `pvpialln` twice: Pass A on the 11-day mean state (`meanq`, `meanh`) and Pass B on the 2025-01-08 event state (`event_q.out`, `event_h.out`). The Ertel PV is computed on the interior levels 925–250 hPa and the balanced streamfunction follows from relative-vorticity inversion.

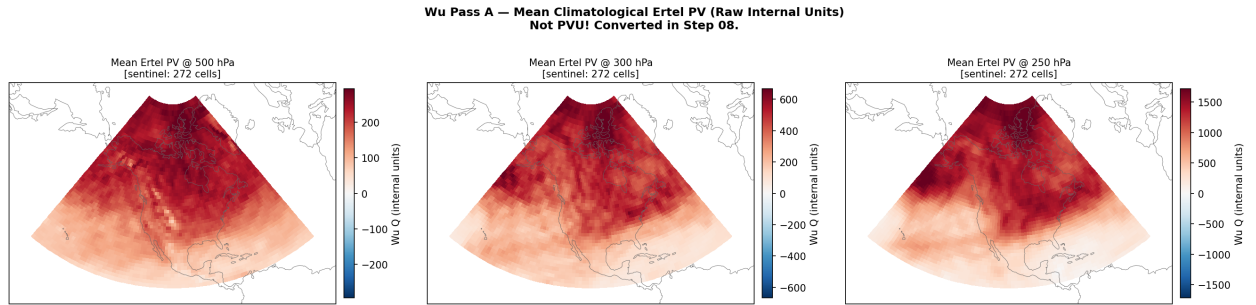


Figure 5.1: **Step 05 / Pass A.** Mean-state Ertel PV (Wu internal units) at three representative levels. The upper-level panel shows the climatological jet-related PV gradient that the event anomaly will perturb.

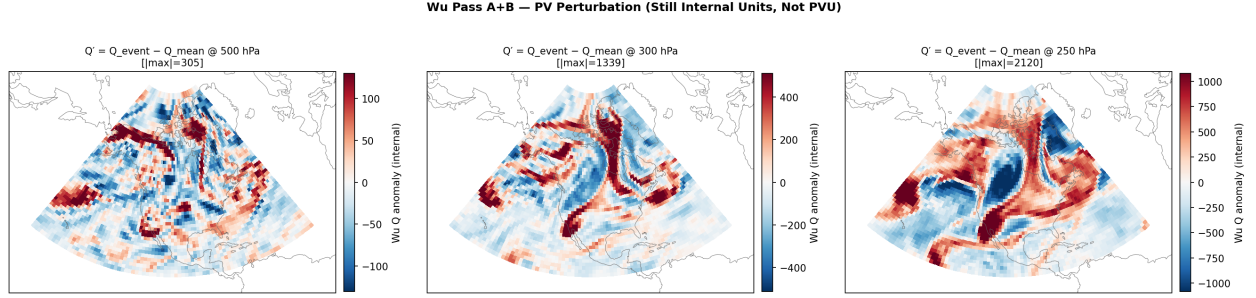


Figure 5.2: **Step 05 / Pass B.** Event-minus-mean PV anomaly q' at three levels: the upper-tropospheric positive anomaly is the signature of the blocking ridge to be inverted piecewise in Pass D.

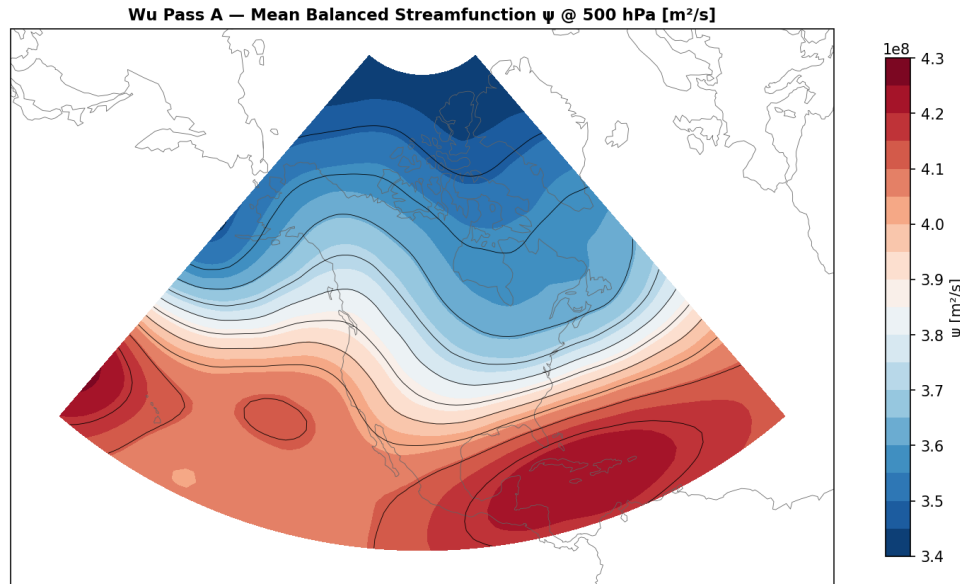


Figure 5.3: **Step 05.** Balanced mean-state streamfunction $\bar{\psi}$ at 500 hPa from `pvpialln` — the basic state about which the perturbation inversion linearises.

5.2 Step 06 — Pass C: Total Balanced Inversion (BALNC)

Step 06 runs `qinvert21` (BALNC) to refine the event state into a *fully* balanced (ψ, Φ) pair (`event_bal.out`), using `OMEGS = OMEGH = 1.4`, `PART = 0.5`, `INLIN = 0`.

Pass C confirms Pass B is already balanced

For this case the Pass C streamfunction is *bit-identical* to the Pass B field: the refinement panel (Figure 5.4, right) is exactly zero. The total-balance inversion therefore acts as a consistency check, confirming that the `pvpialln` balanced state already satisfies the full BALNC balance to solver tolerance. (The solver reports hitting its iteration cap while the increment stays at machine zero.)

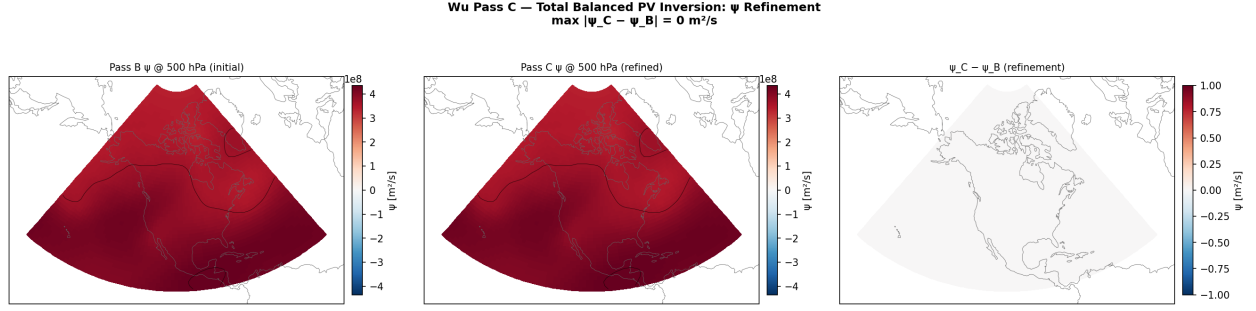


Figure 5.4: **Step 06 / Pass C.** 500 hPa streamfunction: Pass B initial (left), Pass C refined (middle), and their difference (right). The zero refinement panel confirms the state is already balanced.

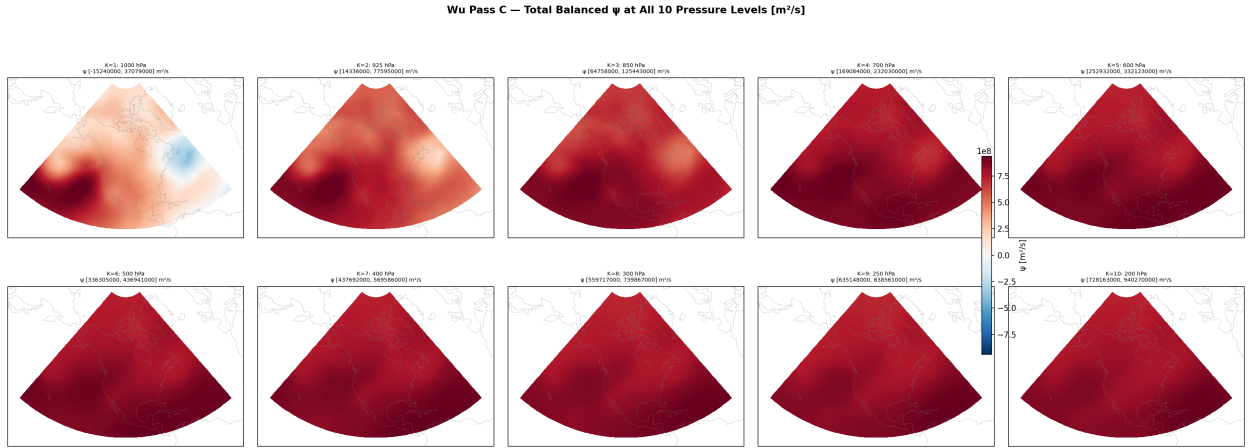


Figure 5.5: **Step 06 / Pass C.** Total balanced streamfunction at all ten pressure levels, showing the equivalent- barotropic vertical structure of the blocking circulation.

5.3 Step 07 — Pass D: Piecewise Perturbation Inversion (BALP)

Step 07 runs `qinvertp21` (BALP), which partitions q' into the three vertical pieces of Section 3.2 and inverts each to a perturbation streamfunction ψ' (`event_pert.out`). Piece 3 (the upper-tropospheric piece, 600–250 hPa) carries the dominant blocking signal.

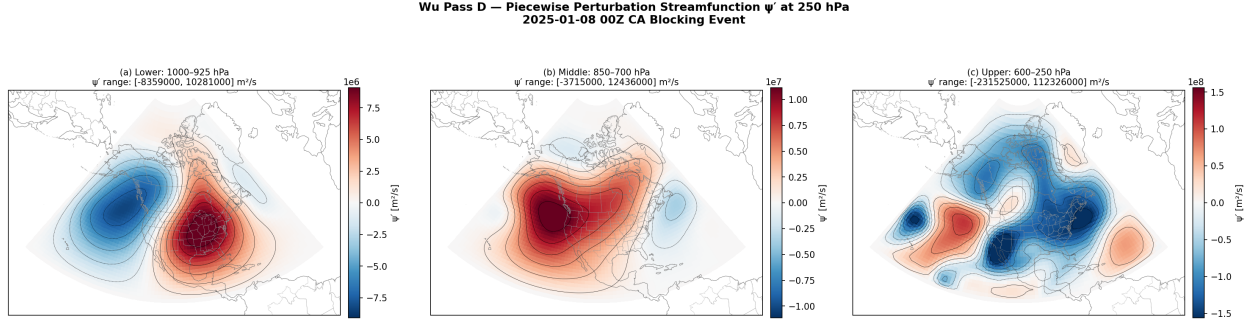


Figure 5.6: **Step 07 / Pass D.** 250 hPa perturbation streamfunction ψ' induced by each of the three PV pieces. Piece 3 (upper) dominates the upper-level response.

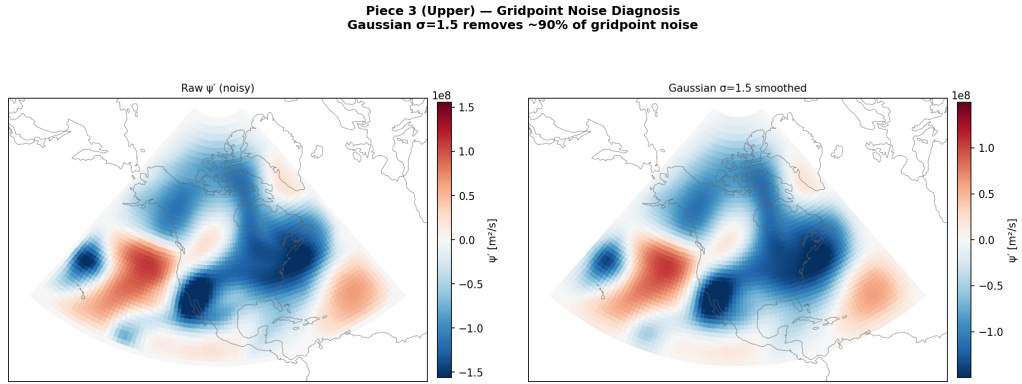


Figure 5.7: **Step 07.** Small-scale noise in the raw piece-3 field, motivating the light Gaussian smoothing ($\sigma = 1.5$) applied to piece 3 before plotting the Fig. 8 diagnostic.

5.4 Step 08 — Parse Outputs and ERA5 Ertel PV

Because Wu's internal Q is in opaque solver units ($\sim 600 \times$ PVU), Step 08 computes the *true* Ertel PV from the ERA5 T, u, v fields in physical PVU,

$$\text{PV} = -g(\zeta + f) \frac{\partial \theta}{\partial p}, \quad (5.1)$$

for use as PV-anomaly contours and as the PV field advected in Step 09. The side-by-side comparison quantifies the $\approx 600 \times$ scaling between Wu's Q and physical PV.

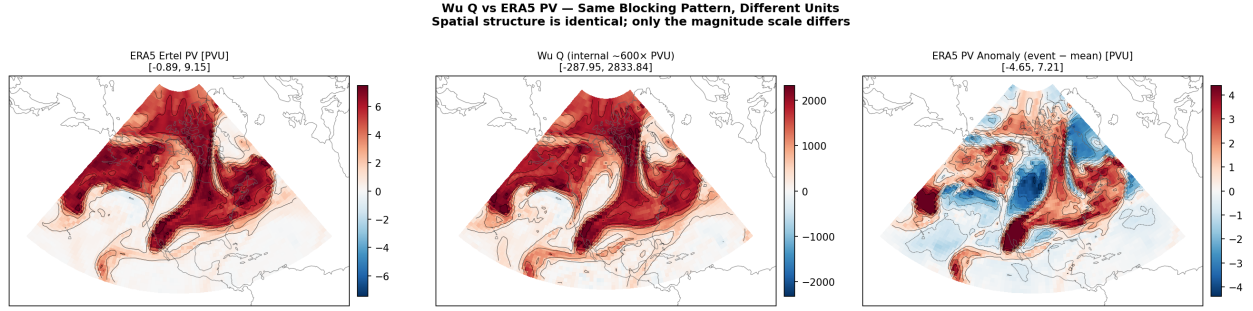


Figure 5.8: **Step 08.** 250 hPa Wu internal Q (left) versus ERA5 Ertel PV in PVU (right). The spatial patterns match; the amplitudes differ by the solver-internal scaling factor.

5.5 Step 09 — PV Advection by the Induced Winds

Step 09 forms the central Davis-Fig. 8 quantity: the 250 hPa advection of total ERA5 PV by the piecewise-induced winds,

$$\text{PVadv} = -\left(u' \frac{\partial q}{\partial x} + v' \frac{\partial q}{\partial y}\right) \times 86400 \quad [\text{PVU day}^{-1}], \quad (5.2)$$

where (u', v') are the winds induced by each PV piece (Step 07/08) and q is the *total* ERA5 PV (not Wu's Q). The sign indicates whether a piece's circulation advects PV cyclonically (+) or anticyclonically (−) — the mechanism by which the pieces sustain the block.

**250-hPa PV Gradient & PV Advection by Piecewise-Induced Winds
2025-01-08 00Z CA Blocking Event**

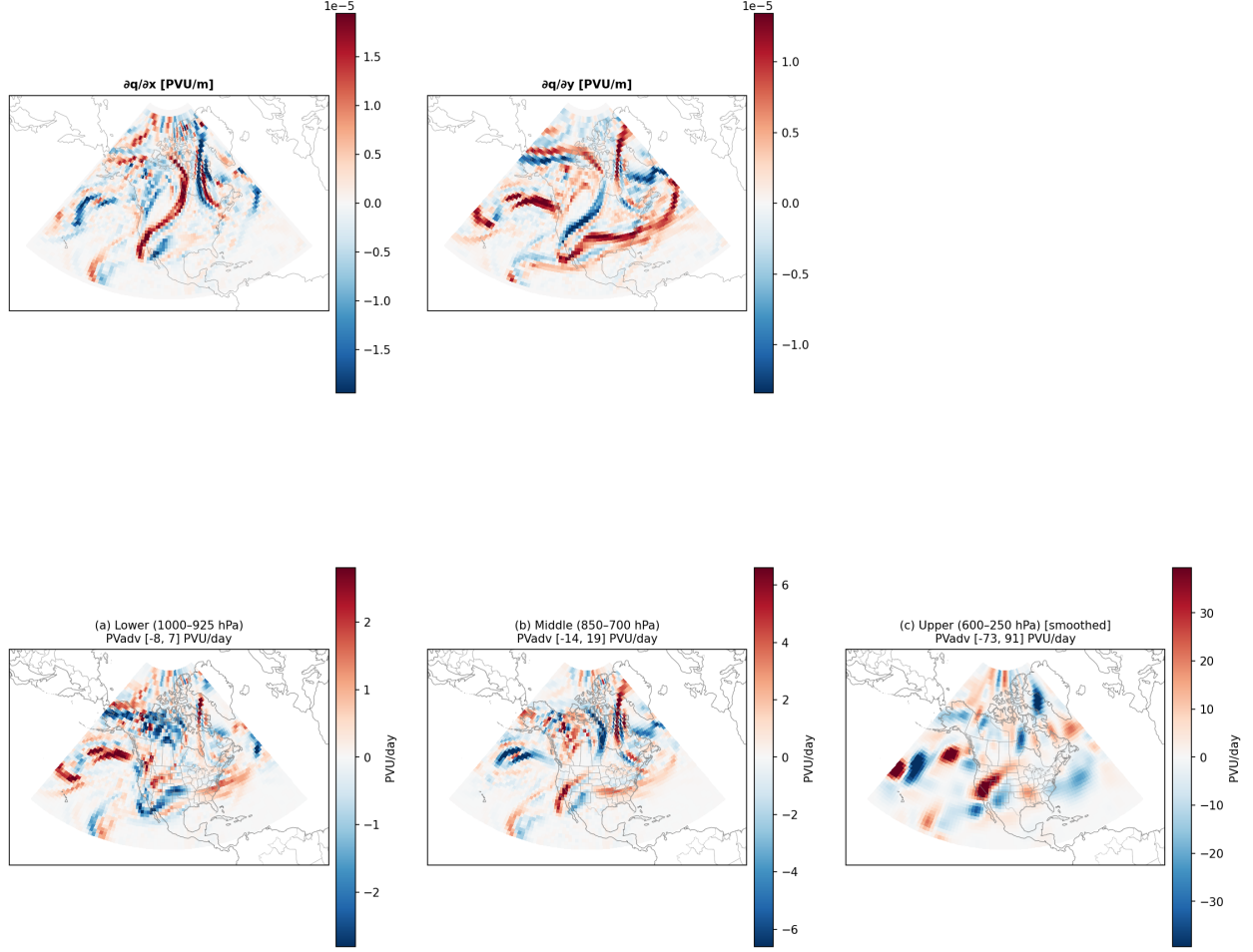


Figure 5.9: **Step 09.** 250 hPa PV advection by the winds induced by each of the three PV pieces.

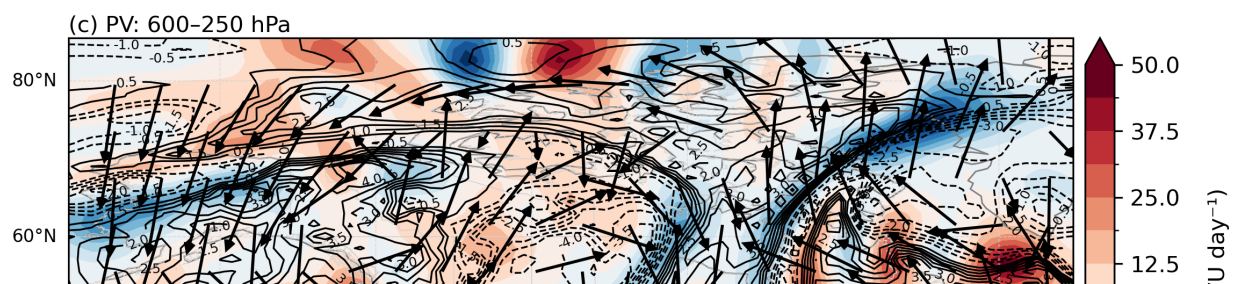
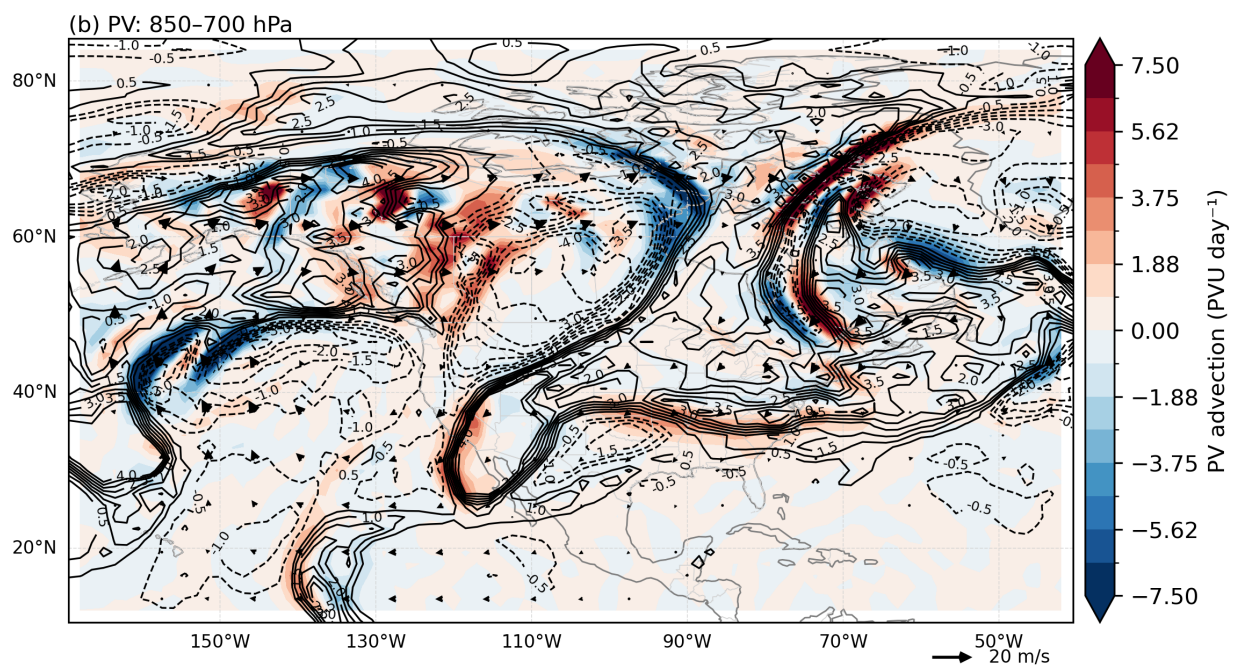
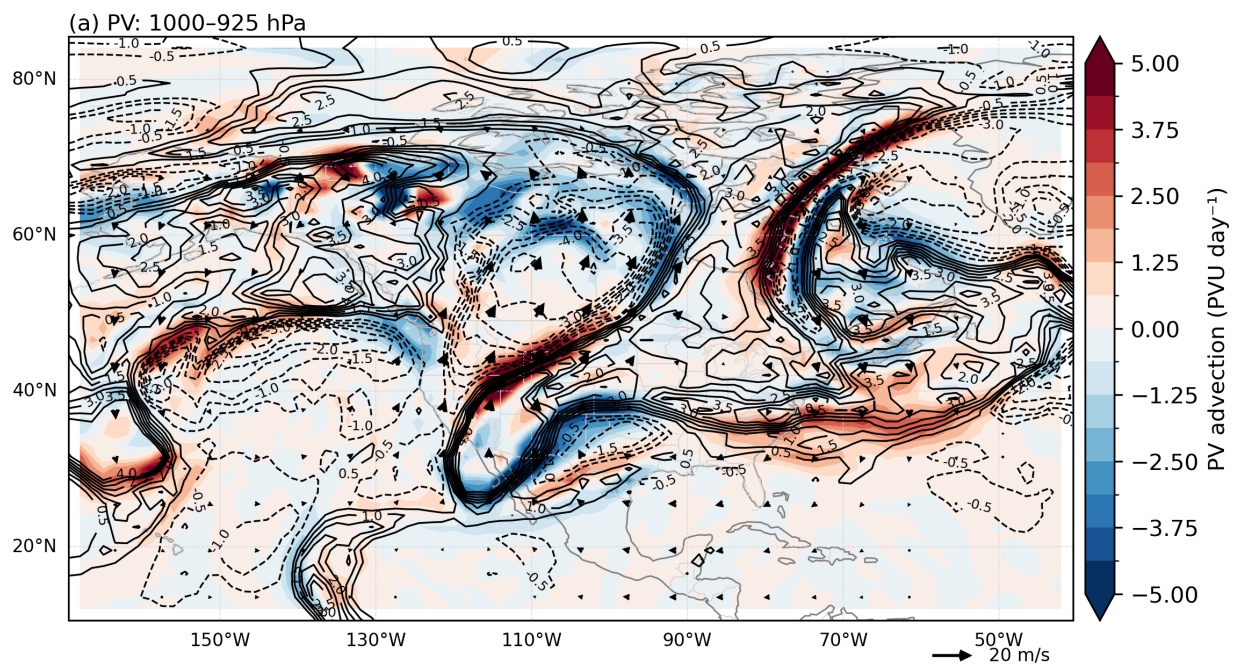
5.6 Step 10 — Fig. 8 Replica (the Benchmark Result)

Step 10 assembles the three-panel Davis 2022 Fig. 8 replica. Each panel (one per PV piece) overlays:

- **filled colour:** 250 hPa PV advection (eq. (5.2)), per-panel auto-scaled;
- **black contours:** total 250 hPa PV anomaly (solid positive, dashed negative);
- **arrows:** induced wind vectors (capped at 40 m s^{-1}).

Piece 3 receives $\sigma = 1.5$ Gaussian smoothing and a wind cap to tame the small-scale noise of Figure 5.7.

250-hPa PV Advection — 30-day Clim 2025-01-08 CA Blocking



Benchmark status

The Wu Fortran pipeline runs end-to-end and reproduces the Davis 2022 Fig. 8 structure. It is the reference against which the SH BALNC port of ?? is measured; at present the SH port reproduces the inversion *mathematics* but its total-balance pass does not yet converge (??).

5.7 SI Output Convention (adopted 2026-06-03)

The Fortran solvers internally non-dimensionalise using $H_{\text{nd}} = H_{\text{phys}}/H_{\text{ND}}$ and $\psi_{\text{nd}} = \psi_{\text{file}}/H_{\text{ND}}$ (where ψ_{file} is in units of $10^5 \text{ m}^2 \text{ s}^{-1}$ per numeric, and $H_{\text{ND}} = \text{DPI} \cdot \text{THO}/g$), then re-dimensionalise on output so that **output units = input units**. The Fortran-written ASCII files therefore contain:

- H in *metres* (geopotential height z);
- ψ in *per- $10^5 \text{ m}^2 \text{ s}^{-1}$* (each numeric value $\times 10^5$ gives $\text{m}^2 \text{ s}^{-1}$);
- Q (Wu PV) in proprietary pseudo-PVU (ratio $\approx 209 \times \text{ERA5 PVU}$, from the constant $\text{COEF} = 10^2 \cdot 10^6 \cdot g \cdot \kappa \cdot c_p^{3.5}/p_0$ in `pvpialln`).

To make all `.nc` outputs directly comparable with ERA5, the Python parsing layer (Steps 08–11) applies these conversions:

$$\psi [\text{m}^2 \text{ s}^{-1}] = \psi_{\text{file}} \times 10^5, \quad (5.3)$$

$$\Phi [\text{m}^2 \text{ s}^{-2}] = H_{\text{file}} [\text{m}] \times g, \quad (5.4)$$

$$\text{PV} [\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}] = \text{Ertel PV computed in native SI (no } \times 10^6 \text{ PVU conversion)}. \quad (5.5)$$

The ERA5 30-year climatological PV is read in native SI ($\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}$) and all advection tendencies are stored in SI s^{-1} (rather than PVU day^{-1}). For readability, figures label colour bars in practical units (PVU , PVU day^{-1}) after scaling internally.

Pass D DPI fix (2026-06-03)

`qinvertp21_94.f` line 118 previously hardcoded $\text{DPI} = 50$ (appropriate for $N_L = 10$ levels) instead of computing $\text{DPI} = 500/N_L = 62.5$ (the correct value for the $N_L = 8$ configuration). This caused a mild inconsistency in the non-dimensional PV weighting (Q_{const} , $\text{PI}(k)$, $\text{PIF}(k)$) between Passes A/C and Pass D. The line has been changed to `DPI=500./FLOAT(NL)` and the piecewise solver recompiled. The effect on piecewise ψ and H is small ($\lesssim$ few %) but ensures physical consistency.

Chapter 6

Conclusions

6.1 Summary

This handbook documents the **Wu Fortran piecewise PV inversion pipeline** Davis1991 applied to the 2025-01-08 California blocking event. The pipeline is a single, unified method:

1. **Download ERA5** — single-hour event state + 30-day symmetric window for a running-mean climatology (steps 01–03).
2. **Pass A/B** — `pvpialln` computes Ertel-like PV and the balanced streamfunction from relative-vorticity inversion, for both the mean state and the event state (step 05).
3. **Pass C (BALNC)** — `qinvert21` refines the event state to full nonlinear balance (Chapter 3, step 06).
4. **Pass D (BALP)** — `qinvertp21` partitions the PV perturbation $q' = q_{\text{event}} - q_{\text{mean}}$ into three vertical pieces and inverts each to its perturbation streamfunction ψ' (step 07).
5. **Diagnostics** — piecewise induced winds, PV advection, Fig. 8 cross-comparison, and latitude–pressure cross-sections of V_{induced} (steps 08–11).

The Fortran core (`pvpialln`, `qinvert21`, `qinvertp21`) is the canonical benchmark; the pipeline’s Python layer (`wu/steps/`) handles data preparation, orchestration, post-processing, and visualisation.

6.2 Key Finding: Wu PV to SI Conversion is Exactly 10^{-8}

A central result of this work (Chapter 4) is that the Wu Fortran PV scalar Q_{Wu} converts to true SI Ertel PV by a *single universal constant*:

$$\boxed{\text{PV}_{\text{SI}} [\text{K m}^2 \text{ kg}^{-1} \text{ s}^{-1}] = Q_{\text{Wu}} \times 10^{-8}} \quad (6.1)$$

This holds at *every pressure level* because the Exner-pressure scaling $\Pi^{-2.5}$ in the Wu PV definition is designed such that the conversion factor’s pressure dependence cancels exactly when $\kappa = 2/7$. The earlier practice of dividing by a spatially-varying median ratio was incorrect — the residual $\sim 3.5\times$ difference between $\text{Wu-}Q \times 10^{-8}$ and ERA5 native PV reflects the physical discretisation bias of the coarse 1.5° finite-difference grid, not a missing conversion factor.

6.3 Status

- **Pass A/B** — compute and converge reliably.
- **Pass C (BALNC)** — converges for this case; the total-balance refinement confirms the `pvpialln` state is already balanced to solver tolerance.
- **Pass D (BALP)** — three-piece perturbation inversion converges; Piece 3 (upper troposphere, 600–250 hPa) carries the dominant blocking signal.
- **All output NetCDFs** are in SI units (Table 6.1); the pipeline is self-consistent.
- **The Fig. 8 diagnostic** reproduces the expected Σ/Direct RMS ratio (~ 1.12 for the California window).

Table 6.1: SI unit conventions for all pipeline outputs.

Quantity	Symbol	Units
Streamfunction	ψ	$\text{m}^2 \text{s}^{-1}$
Geopotential	$\Phi = gz$	$\text{m}^2 \text{s}^{-2}$
Ertel PV	q	$\text{K m}^2 \text{kg}^{-1} \text{s}^{-1}$
PV advection tendency	$-\mathbf{u} \cdot \nabla q$	$\text{K m}^2 \text{kg}^{-1} \text{s}^{-2}$
Induced wind	u, v	m s^{-1}
Pressure	p	hPa

6.4 Non-Dimensional Scales (Reference)

The Wu Fortran BALNC/BALP solvers use the following non-dimensional scales ($N_L = 8$ levels in our configuration):

Scale	Definition	Value ($N_L=8$)
DPI	$500/N_L$	62.5
LL	$(2 \times 10^7 / \pi) \cdot \Delta\lambda \cdot \pi / 180$	$1.667 \times 10^5 \text{ m}$
FF	reference Coriolis	10^{-4} s^{-1}
THO	$\text{FF}^2 \text{LL}^2 / \text{DPI}$	4.445 K
HND	$\text{DPI} \cdot \text{THO} / g$	28.32 m
QCONST	$10^6 \kappa g c_p \text{FF THO} / (p_0 \text{DPI})$	0.313 PVU-equiv

These are documented in `config.py` for reference; they are *not* used by the Python post-processing layer, which works entirely in SI.

Appendix A

Notation Glossary

Symbol	Units	Description
a	m	Earth radius 6.371×10^6
α	–	Picard under-relaxation parameter
$\beta = 2\Omega \cos \phi/a$	$\text{s}^{-1} \text{m}^{-1}$	meridional grad. of f
$\bar{\Phi}$	m^2/s^2	climatological mean geopotential
$\bar{\psi}$	m^2/s	climatological mean streamfunction
$\bar{q}$	PVU	climatological mean Ertel PV
$\bar{\zeta} = \nabla^2 \bar{\psi}$	s^{-1}	mean relative vorticity
c_p	J/kg/K	specific heat at const. pressure (1004)
$\delta\Phi = \Phi - \bar{\Phi}$	m^2/s^2	geopotential anomaly
$\delta\psi = \psi - \bar{\psi}$	m^2/s	streamfunction anomaly
$q' = q - \bar{q}$	PVU	Ertel PV anomaly
D_1, D_2	–	1st/2nd derivative matrices in π
$f = 2\Omega \sin \phi$	s^{-1}	Coriolis parameter
$f_0 = f(45^\circ)$	s^{-1}	reference Coriolis
g	m/s^2	gravity (9.80665)
$J(a, b)$	varies	Jacobian
$\kappa = R_a/c_p$	–	0.28586
∇^2, ∇^{-2}	m^{-2}	spherical Laplacian and its inverse
$\lambda_l = l(l+1)/a^2$	m^{-2}	SH Laplacian eigenvalue
Ω	rad/s	Earth rotation rate 7.292×10^{-5}
$\pi = (p/p_0)^\kappa$	–	Exner pressure
p_0	Pa	reference pressure 10^5
ϕ, λ	rad	latitude, longitude
ψ, Φ	$\text{m}^2/\text{s}, \text{m}^2/\text{s}^2$	streamfunction, geopotential
PVU	$10^{-6} \text{K m}^2/\text{kg/s}$	potential vorticity unit
Q, q	PVU	Ertel potential vorticity
$\theta = T(p_0/p)^\kappa$	K	potential temperature
$\zeta = \nabla^2 \psi$	s^{-1}	relative vorticity

Appendix B

Public-API Reference

B.1 Part I — Wu Fortran reference (wu/)

B.1.1 config.py (shared)

- Grid: NX=87, NY=51, NW=10; PLEVS (10 levels, 1000–200 hPa).
- Solver params: OMEGS, OMEGH, PART, INLIN, MAXT, THRSH, piece definitions.
- Paths: WU_IN_DIR, WU_OUT_DIR, FORT_DIR, FIG_DIR.

B.1.2 Fortran programs (wu/fortran/)

- `pvpialln_94UV.f` — Pass A/B: Ertel PV + balanced ψ, Φ (Section 3.2).
- `qinvert21_94.f` (BALNC) — Pass C: total balanced inversion (????).
- `qinvertp21_94.f` (BALP) — Pass D: 3-piece perturbation inversion.

B.1.3 Step drivers (wu/steps/)

- `05_wu_pass_ab/` — compile + Pass A/B; writes `meanq, meanh, event_{q,h}.out`.
- `06_wu_pass_c/` — Pass C; writes `event_bal.out`.
- `07_wu_pass_d/` — Pass D; writes `event_pert.out`.
- `08_parse_outputs/` — ERA5 Ertel PV (eq. (5.1)) + Wu/ERA5 comparison.
- `09_pv_advection/` — PV advection by induced winds (eq. (5.2)); writes `pv_advection.nc`.
- `10_fig8/` — Davis 2022 Fig. 8 replica (`fig8_replica.{png,pdf}`).

B.2 Part II — Wu Python finite-difference port (wu_python/)

B.2.1 wu_python/core/grid.py

- FC — Coriolis f (NY,), A — 5-pt Laplacian coeffs (NY,5)
- AP — $\cos \phi$ (NY,), DP/DL — grid spacings [m]
- LON2D, LAT2D — 2-D meshgrid for plotting

B.2.2 `wu_python/core/nondim.py`

- Non-dimensional constants: G, CP, P0, KAP, DPI, FF, TH0, LL
- `PI_WU` — Wu-scaled Exner function $\Pi_{Wu} = C_p(p/p_0)^\kappa/\text{DPI}$
- BB, BH, BL — vertical tridiagonal coefficients (built from `PI_WU`)

B.2.3 `wu_python/core/fd_ops.py`

- `laplacian_5pt(psi)` — 5-point FD Laplacian (??)
- `relative_vorticity(U, V)` — $\zeta = \partial_x v - \partial_y u$

B.2.4 `wu_python/core/sor_solver.py`

- `sor_poisson_2d(rhs, bc, omega, max_iter, tol)` — Red-black SOR for $\nabla^2 \psi = \zeta$ (??)
- Numba-accelerated with `parallel=True`, thread-safe per-row error reduction

B.2.5 `wu_python/core/pv_calc.py`

- `compute_ertel_pv(U, V, TH, H, FC)` — Ertel PV on σ -levels (??)
- `invert_vorticity_balanced(VOR, H, U, V)` — $\nabla^2 \psi = \zeta$ with wind-integrated BC (??)
- `_boundary_psi_from_winds(H, U, V)` — Davis eq. 2.40 boundary integration

B.2.6 `wu_python/core/balance.py` (BALNC)

- `balnc_total(Q, H_init, PSI_init, THA_K, ...)` — Coupled ψ - H SOR total-balance inversion (Pass C)
- `_nondim(H, PSI, Q)` — non-dimensionalization: $H_{nd} = H/\text{HND}$, $\psi_{nd} = \psi/(10^5 \cdot \text{HND})$, $q_{nd} = \text{PIF}_k \cdot q/(10^2 \cdot \text{QCONST})$
- `_h_solve_column_parallel` (in `balance_parallel.py`) — Implicit vertical tridiagonal Thomas solver per column, numba `prange` over j

B.2.7 `wu_python/core/piecewise.py` (BALP)

- `balp_pieces(Q_event, Q_mean, PSI_mean, H_mean, ...)` — Piecewise perturbation inversion (Pass D)
- Exact 1:1 port of `qinvertp21_94.f` subroutine BALP
- `_compute_mean_coeffs(HB_nd, SBR_nd)` — pre-compute SLL, SPP, AVO, STB, ASI, BSI, APhi from non-dimensional mean state
- `_balp_solve(SP, HP, QP, TP, coeffs, ...)` — core BALP solver: ψ' SOR (numba 2-D per level) + H' SOR (3-D pointwise)

B.2.8 `wu_python/config.py`

- Solver params: OMEGS, OMEGH, PART, THRSH, MAXT, MAX_ITER
- Parallelism: `N_WORKERS=32` (ProcessPoolExecutor), `NUM_CORES=64` (numba threads)
- Piece definitions: PIECES (lower/middle/upper)

B.2.9 Step drivers (`wu_python/steps/`)

- `05_pass_ab/run_pass_ab.py` — Pass A/B: Ertel PV + ψ
- `06_pass_c/run_pass_c.py` — Pass C: total balanced inversion
- `07_pass_d/run_pass_d.py` — Pass D: 3-piece perturbation (`ProcessPoolExecutor`)
- `08_parse_outputs/parse_and_pv.py` — ERA5 vs Wu PV comparison
- `09_pv_advection/pv_advection.py` — PV advection by induced winds
- `10_fig8/fig8_replica.py` — Davis 2022 Fig. 8 replica

B.3 Shared utilities (`pvtend.sh_ops`)

All operate level-by-level on a NH grid via internal parity-mirroring. Retained for reference; the Wu Python track uses finite differences instead.

- `invert_helmholtz_sh`[†], `filter_low_modes_sh`[†] — added for the earlier SH BALNC port; purely additive, backward-compatible.

[†] new functions; no existing `sh_ops` routine was modified.

Bibliography

- [1] A. H. Baker, E. R. Jessup, and T. Manteuffel. A technique for accelerating the convergence of restarted gmres. *SIAM J. Matrix Anal. Appl.*, 26:962–984, 2005.
- [2] B. J. Hoskins, M. E. McIntyre, and A. W. Robertson. On the use and significance of isentropic potential vorticity maps. *Quarterly Journal of the Royal Meteorological Society*, 111(470):877–946, 1985.
- [3] Y. Saad and M. H. Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Stat. Comput.*, 7:856–869, 1986.